Notebook demo reference

Not Every Notebook Needs Spark: SQL-First Analytics with Pure Python

Jax Data User Group | October 21, 2026

Use this reference alongside the downloadable notebooks. It includes the five core notebooks and the optional Polars SQL exercise, with code and explanations for working through the examples in your own Fabric workspace.

Before you start

Import the notebooks into Microsoft Fabric as Pure Python notebooks. Attach your own schema-enabled Lakehouse named lh_pond. Run 0. Setup with DATA_SCOPE = "one_year", then notebooks 1-4 in order. The optional Polars exercise starts in a fresh Python session after setup. These examples use Fabric notebookutils and the Lakehouse mount; they are not standalone local Python scripts.

Setup downloads the 2025 flight files and reference data, writes derived files and replaces dbo.flights_202501. Optional Delta publishing, shortcut and private Blob examples are disabled until configured. Replace private-storage placeholders with your own resource names. Use your workspace settings to choose the Python node size.

Notebook order

0. Setup

1. Query a Lakehouse File

2. Friendly SQL

3. CSV to Parquet

4. Query Data Where It Lives

5. Polars SQL Encore

0. Setup

Setup: prepare the Lakehouse demo data

Run this notebook before notebooks 1-4. Use Pure Python and attach a schema-enabled Lakehouse named lh_pond.

The standard demo uses 2025 flight Parquet files and airport reference CSVs. Later setup cells create a Delta table and an optional Excel workbook.

1. Choose the data scope

Use DATA_SCOPE = "one_year" for this session. It selects January-December 2025 and prints the workspace, Lakehouse and output paths.

full prepares 2022-2025 for separate validation; the attendee demos still read one_year.

```
from datetime import datetime
from pathlib import Path
from urllib.error import HTTPError, URLError
from urllib.request import urlretrieve
import csv

import duckdb
import notebookutils

DATA_SCOPE = "one_year"  # Standard session data; choose "full" for separate validation.

DATA_RANGES = {
    "one_year": ("2025-01", "2025-12"),
    "full": ("2022-01", "2025-12"),
}

if DATA_SCOPE not in DATA_RANGES:
    valid_scopes = ", ".join(sorted(DATA_RANGES))
    raise ValueError(f"DATA_SCOPE must be one of: {valid_scopes}")

START_MONTH, END_MONTH = DATA_RANGES[DATA_SCOPE]

LAKEHOUSE = "/lakehouse/default"
OUTPUT_DIR = Path(f"{LAKEHOUSE}/Files/flights_eu/{DATA_SCOPE}")
REFERENCE_DIR = Path(f"{LAKEHOUSE}/Files/flights_eu/reference")
SCHEMA_DRIFT_DIR = OUTPUT_DIR / "schema_drift"
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
REFERENCE_DIR.mkdir(parents=True, exist_ok=True)
SCHEMA_DRIFT_DIR.mkdir(parents=True, exist_ok=True)

OPDI_URL_TEMPLATE = (
    "https://www.eurocontrol.int/performance/data/download/"
    "OPDI/v002/flight_list/flight_list_{yyyymm}.parquet"
)
AIRPORTS_URL = "https://davidmegginson.github.io/ourairports-data/airports.csv"
AIRLINES_URL = "https://raw.githubusercontent.com/jpatokal/openflights/master/data/airlines.dat"
AIRLINES_COLUMNS = [
    "airline_id",
    "name",
    "alias",
    "iata_code",
    "icao_code",
    "callsign",
    "country",
    "active",
]

print("Data scope:", DATA_SCOPE)
print("Date range:", START_MONTH, "to", END_MONTH)
print("Workspace:", notebookutils.runtime.context["currentWorkspaceName"])
print("Lakehouse:", notebookutils.runtime.context["defaultLakehouseName"])
print("Output:", OUTPUT_DIR)
print("Reference data:", REFERENCE_DIR)
print("Schema-drift demo data:", SCHEMA_DRIFT_DIR)
```

2. Define the download helpers

Define the functions; downloads start in the next cell. Existing non-empty files are skipped, and failed downloads are removed.

The airline helper converts the downloaded reference file into a CSV with column names.

```
def month_range(start: str, end: str):
    start_dt = datetime.strptime(start, "%Y-%m")
    end_dt = datetime.strptime(end, "%Y-%m")
    if end_dt < start_dt:
        raise ValueError(f"END_MONTH ({end}) is before START_MONTH ({start})")

    year, month = start_dt.year, start_dt.month
    while (year, month) <= (end_dt.year, end_dt.month):
        yield f"{year}{month:02d}"
        month += 1
        if month > 12:
            month = 1
            year += 1


def download(url: str, dest: Path) -> bool:
    if dest.exists() and dest.stat().st_size > 0:
        print(f"skip: {dest.name}")
        return True

    try:
        print(f"download: {dest.name} ...", end=" ", flush=True)
        urlretrieve(url, dest)
        size_mb = dest.stat().st_size / (1024 * 1024)
        print(f"done ({size_mb:.1f} MB)")
        return True
    except HTTPError as error:
        print(f"failed HTTP {error.code}: {url}")
    except URLError as error:
        print(f"failed {error.reason}: {url}")

    if dest.exists():
        dest.unlink()
    return False


def download_airlines(dest: Path) -> bool:
    if dest.exists() and dest.stat().st_size > 0:
        print(f"skip: {dest.name}")
        return True

    raw_dest = dest.with_suffix(".dat")
    if not download(AIRLINES_URL, raw_dest):
        return False

    with open(raw_dest, newline="", encoding="utf-8") as source_file:
        rows = list(csv.reader(source_file))

    with open(dest, "w", newline="", encoding="utf-8") as target_file:
        writer = csv.writer(target_file)
        writer.writerow(AIRLINES_COLUMNS)
        for row in rows:
            writer.writerow(["" if value == r"\N" else value for value in row])

    raw_dest.unlink(missing_ok=True)
    print(f"converted: {raw_dest.name} -> {dest.name}")
    return True
```

3. Download the flight and reference files

Download the selected months and shared lookup CSVs into Lakehouse Files. The cell stops with a list of missing files if a download fails.

Flight files go into Files/flights_eu/{DATA_SCOPE}; reference files go into Files/flights_eu/reference.

```
months = list(month_range(START_MONTH, END_MONTH))
print(f"Pulling {len(months)} month(s): {START_MONTH} -> {END_MONTH}")
print()
```

```
failures = []
for yyyymm in months:
    url = OPDI_URL_TEMPLATE.format(yyyymm=yyyymm)
    dest = OUTPUT_DIR / f"flight_list_{yyyymm}.parquet"
    if not download(url, dest):
        failures.append(yyyymm)

print()
print("Reference data:")
airports_ok = download(AIRPORTS_URL, REFERENCE_DIR / "airports.csv")
airlines_ok = download_airlines(REFERENCE_DIR / "airlines.csv")

if failures:
    raise RuntimeError(f"Missing months: {', '.join(failures)}")
if not airports_ok or not airlines_ok:
    missing_references = []
    if not airports_ok:
        missing_references.append("airports.csv")
    if not airlines_ok:
        missing_references.append("airlines.csv")
    raise RuntimeError(f"Missing reference data: {', '.join(missing_references)}")

print("Setup download complete.")
```

4. Verify the download and count flight rows

Check that every expected monthly file exists, then count the rows with DuckDB. Missing or unexpected month files stop the check.

This verifies file names, non-empty reference files and queryability; it is not a full data-quality audit.

```
con = duckdb.connect()
flight_files = f"{OUTPUT_DIR}/flight_list_*.parquet"
expected_filenames = {
    f"flight_list_{yyyymm}.parquet"
    for yyyymm in month_range(START_MONTH, END_MONTH)
}
actual_filenames = {path.name for path in OUTPUT_DIR.glob("flight_list_*.parquet")}
assert actual_filenames == expected_filenames, (
    "Flight files do not match the selected scope. "
    f"Missing: {sorted(expected_filenames - actual_filenames)}; "
    f"unexpected: {sorted(actual_filenames - expected_filenames)}"
)
assert (REFERENCE_DIR / "airports.csv").stat().st_size > 0, "airports.csv is missing or empty"
assert (REFERENCE_DIR / "airlines.csv").stat().st_size > 0, "airlines.csv is missing or empty"

flight_count = con.sql(f"""
    SELECT count(*) AS flights
    FROM read_parquet('{flight_files}')
""").df()

display(flight_count)
```

4a. Check the count for each monthly file

List the row count in each downloaded Parquet file. This makes missing or unexpectedly small months easier to spot.

```
con.sql(f"""
    SELECT filename,
           count(*) AS rows_in_file
    FROM read_parquet('{flight_files}', filename = true)
    GROUP BY 1
    ORDER BY 1
""").df()
```

5. Create the January Delta table for notebook 4

Write January's flight data to dbo.flights_202501. Notebook 4 reads it with delta_scan and compares its Barcelona count with the original Parquet file.

This overwrites that derived table through OneLake. The source files in Files are unchanged.

```
from deltalake import DeltaTable, write_deltalake
```

```
ctx = notebookutils.runtime.context

workspace_id = ctx["defaultLakehouseWorkspaceId"]
lakehouse_id = ctx["defaultLakehouseId"]

assert workspace_id and lakehouse_id, (
    "Attach a default schema-enabled Lakehouse before running this notebook."
)

TABLE_SCHEMA = "dbo"
TABLE_NAME = "flights_202501"

table_path = (
    f"abfss://{workspace_id}@onelake.dfs.fabric.microsoft.com/"
    f"{lakehouse_id}/Tables/{TABLE_SCHEMA}/{TABLE_NAME}"
)

storage_options = {
    "bearer_token": notebookutils.credentials.getToken("storage"),
    "use_fabric_endpoint": "true",
}

january_file = OUTPUT_DIR / "flight_list_202501.parquet"
assert january_file.exists(), f"January source file is missing: {january_file}"

january_flights = con.sql(f"""
    SELECT adep,
           ades,
           typecode,
           cast(dof AS varchar) AS dof
    FROM read_parquet('{january_file}')
""").arrow()

write_deltalake(
    table_path,
    january_flights,
    mode="overwrite",
    storage_options=storage_options,
)

delta_table = DeltaTable(table_path, storage_options=storage_options)

print(
    f"Created Delta table: {TABLE_SCHEMA}.{TABLE_NAME} "
    f"(version {delta_table.version()})"
)
```

6. Optional exercise: create schema-drift examples

Create two small Parquet exports; the second adds source_batch. The displayed UNION ALL BY NAME aligns their columns and fills the missing field with NULL.

This is an extra setup exercise, not a dependency of notebook 2's examples. It replaces only these derived fixture files.

```
schema_drift_files = {
    "earlier": SCHEMA_DRIFT_DIR / "flights_earlier.parquet",
    "later": SCHEMA_DRIFT_DIR / "flights_later.parquet",
}

for path in schema_drift_files.values():
    path.unlink(missing_ok=True)

con.sql(f"""
    COPY (
        SELECT adep,
               ades,
               typecode,
               dof
         FROM read_parquet('{flight_files}')
        WHERE adep IS NOT NULL
          AND ades IS NOT NULL
        LIMIT 500
    ) TO '{schema_drift_files["earlier"]}' (FORMAT PARQUET)
""")
```

```
con.sql(f"""
    COPY (
        SELECT adep,
               ades,
               typecode,
               dof,
               '2025-refresh' AS source_batch
        FROM read_parquet('{flight_files}')
        WHERE adep IS NOT NULL
          AND ades IS NOT NULL
        LIMIT 500
    ) TO '{schema_drift_files["later"]}' (FORMAT PARQUET)
""")

assert all(path.exists() for path in schema_drift_files.values()), (
    "Schema-drift fixture files were not created."
)

con.sql(f"""
    SELECT *
    FROM (
        SELECT *
        FROM read_parquet('{schema_drift_files["earlier"]}')
        LIMIT 3
    )

    UNION ALL BY NAME

    SELECT *
    FROM (
        SELECT *
        FROM read_parquet('{schema_drift_files["later"]}')
        LIMIT 3
    )
""").df()
```

7. Create the workbook for notebook 4's optional Excel example

Write January's Barcelona departures by destination to an .xlsx file, then read it back. Its flights total should match the other January sources.

Missing destinations are labelled UNKNOWN to keep the first text cell non-empty for type inference. This replaces the derived workbook.

```
con.execute("INSTALL excel")
con.execute("LOAD excel")

EXCEL_DIR = Path(f"{LAKEHOUSE}/Files/flights_eu/excel")
EXCEL_DIR.mkdir(parents=True, exist_ok=True)
excel_file = EXCEL_DIR / "barcelona_departures_202501.xlsx"
excel_file.unlink(missing_ok=True)

con.sql(f"""
    COPY (
        SELECT coalesce(ades, 'UNKNOWN') AS destination,
               count(*) AS flights
        FROM read_parquet('{january_file}')
        WHERE adep = 'LEBL'
        GROUP BY ALL
        ORDER BY flights DESC
    ) TO '{excel_file}' WITH (FORMAT xlsx, HEADER true)
""")

assert excel_file.exists() and excel_file.stat().st_size > 0, "Excel workbook was not created."

con.sql(f"""
    SELECT count(*) AS destinations,
           CAST(sum(flights) AS BIGINT) AS flights_from_barcelona
    FROM read_xlsx('{excel_file}')
""").df()
```

1. Query a Lakehouse File

Query a Lakehouse file with SQL

Use DuckDB inside a Pure Python notebook to explore Parquet without a Spark session.

Attach lh_pond, run 0. Setup with DATA_SCOPE = "one_year", then run sections 0-6 in order. Optional examples follow the resource check.

0. Connect the notebook to its tools

Import DuckDB, then confirm the workspace and attached Lakehouse.

In the next section, we'll set the file path and open a DuckDB connection.

Then we'll query the files with SQL.

Import duckdb and notebookutils

duckdb provides the SQL engine. notebookutils provides access to the Fabric context.

```
import duckdb         # Bring DuckDB into scope to connect and run SQL.
import notebookutils  # Access Fabric utilities, including workspace and Lakehouse context.
```

Confirm the Fabric context

These values come from the notebook's attached Lakehouse. They are useful for orientation only;

DuckDB will query the mounted files in the next steps.

```
# Get the Fabric context for this notebook session.
ctx = notebookutils.runtime.context

# Show the current workspace and the attached default Lakehouse.
print("Workspace:", ctx["currentWorkspaceName"])
print("Lakehouse:", ctx["defaultLakehouseName"])
```

1. Point SQL at the Lakehouse files

FLIGHT_FILES holds the path to our Parquet files in the Lakehouse Files area.

The flight_list_*.parquet pattern selects the monthly files in the one-year folder.

DuckDB's read_parquet() lets SQL query those files directly.

Defining the path does not read or copy the data.

```
# One Year
FLIGHT_FILES = "/lakehouse/default/Files/flights_eu/one_year/flight_list_*.parquet"

# Optional full dataset: 2022-2025
#FLIGHT_FILES = "/lakehouse/default/Files/flights_eu/full/flight_list_*.parquet"


print("Flight files:", FLIGHT_FILES)
```

Open DuckDB

DuckDB is already available in the Pure Python runtime and runs inside this Python

process. This connection uses an in-memory database, while the source files remain

in the Lakehouse.

We can query those files without first loading the full dataset into a pandas DataFrame.

```
# Open an in-memory DuckDB database inside this Python process.
con = duckdb.connect()

# Show the DuckDB version available in this session.
```

```
print("DuckDB:", duckdb.__version__)
```

2. Ask the first SQL question

How many flight rows are in these files?

read_parquet() reads the matching files as one table for this query.

union_by_name = true aligns columns by name across the files.

The f before the SQL string inserts the FLIGHT_FILES path into the query.

DuckDB counts the rows; .show() displays the result.

```
con.sql(f"""
    SELECT count(*) AS flights
    FROM read_parquet('{FLIGHT_FILES}', union_by_name = true)
""").show()
```

About 16.2 million for one year

The optional full setup covers 2022-2025. Run a count to verify the downloaded data.

3. Inspect the column names and types

Use DESCRIBE to see which fields the files expose. It reports schema, not data quality.

dof is stored as text in these files; later examples convert it for date calculations.

```
con.sql(f"""
    DESCRIBE SELECT *
    FROM read_parquet('{FLIGHT_FILES}', union_by_name = true)
""").show()
```

4. Trust, but verify

Real flight data can have missing values. Check coverage before filtering rows or trusting

a count.

count(*) counts every row. count(column) counts rows where that column is not NULL.

Each percentage shows how often a field is populated. Empty strings still count as populated.

```
con.sql(f"""
    SELECT count(*) AS rows_total,
           round(100.0 * count(adep) / count(*), 1) AS origin_present_pct,
           round(100.0 * count(ades) / count(*), 1) AS destination_present_pct,
           round(100.0 * count(typecode) / count(*), 1) AS typecode_present_pct,
           round(100.0 * count(registration) / count(*), 1) AS registration_present_pct
    FROM read_parquet('{FLIGHT_FILES}', union_by_name = true)
""").show()
```

5. Name Barcelona's busiest routes with a CSV join

Join flight Parquet to the airport CSV and show the top ten routes involving Barcelona (LEBL). The CSV supplies names for both ends of each route.

Left joins keep flight rows even when an airport is missing from the reference data.

```
AIRPORTS = "/lakehouse/default/Files/flights_eu/reference/airports.csv"
FLIGHT_FILES = "/lakehouse/default/Files/flights_eu/one_year/flight_list_*.parquet"

con.sql(f"""
    SELECT f.adep,
           coalesce(dep.municipality, dep.name) AS origin,
           f.ades,
           coalesce(arr.municipality, arr.name) AS destination,
           count(*) AS flights
    FROM read_parquet('{FLIGHT_FILES}', union_by_name = true) AS f
    LEFT JOIN read_csv_auto('{AIRPORTS}') AS dep
      ON f.adep = dep.ident
    LEFT JOIN read_csv_auto('{AIRPORTS}') AS arr
```

```
      ON f.ades = arr.ident
    WHERE (f.adep = 'LEBL' OR f.ades = 'LEBL')
      AND f.adep IS NOT NULL
      AND f.ades IS NOT NULL
    GROUP BY f.adep, coalesce(dep.municipality, dep.name),
             f.ades, coalesce(arr.municipality, arr.name)
    ORDER BY flights DESC
    LIMIT 10
""").show()
```

6. Bring only the top 25 airport results into pandas

DuckDB joins and aggregates the files; .df() returns the small result to Python. The full flight dataset is not loaded into a pandas DataFrame.

The inner join includes only airports matched in the reference CSV; grouping by airport ID keeps identical labels separate.

```
airport_departures = con.sql(f"""
    SELECT a.municipality,
           a.name,
           count(*) AS departures
    FROM read_parquet('{FLIGHT_FILES}', union_by_name = true) AS f
    JOIN read_csv_auto('{AIRPORTS}') AS a
      ON f.adep = a.ident
    WHERE f.adep IS NOT NULL
    GROUP BY a.ident, a.municipality, a.name
    ORDER BY departures DESC
    LIMIT 25
""").df()

display(airport_departures)
```

Pause: inspect resource usage

Open the notebook's Resource usage panel.

Check the node size, CPU activity, and memory use during these queries.

Use these measurements to help decide whether the workload still fits on this node.

Optional: publish the airport result as Delta

Set RUN_DELTA_WRITE = True to save section 6's result as dbo.busiest_airports. Both Delta cells are skipped by default.

This requires a schema-enabled Lakehouse and overwrites that derived table through OneLake's abfss:// path.

```
RUN_DELTA_WRITE = False  # Set to True to publish the result.

if RUN_DELTA_WRITE:
    from deltalake import write_deltalake

    workspace_id = ctx["defaultLakehouseWorkspaceId"]
    lakehouse_id = ctx["defaultLakehouseId"]
    assert workspace_id and lakehouse_id, "Attach a default Lakehouse first."

    table_schema = "dbo"
    table_name = "busiest_airports"
    table_path = (
        f"abfss://{workspace_id}@onelake.dfs.fabric.microsoft.com/"
        f"{lakehouse_id}/Tables/{table_schema}/{table_name}"
    )

    storage_options = {
        "bearer_token": notebookutils.credentials.getToken("storage"),
        "use_fabric_endpoint": "true",
    }

    write_deltalake(
        table_path,
        airport_departures,
        mode="overwrite",
        storage_options=storage_options,
    )
    print(f"Wrote Delta table: {table_schema}.{table_name}")
```

```
else:
    print("Delta publishing skipped.")
```

Read back the published Delta table

Show ten rows from the table just written. This cell runs only when RUN_DELTA_WRITE is True; it verifies read-back rather than displaying the original DataFrame.

```
if RUN_DELTA_WRITE:
    from deltalake import DeltaTable

    delta_table = DeltaTable(table_path, storage_options=storage_options)
    display(delta_table.to_pandas().head(10))
```

7. Keep exploring

Use these additional exercises to explore the dataset.

Small, iterative questions often fit on one Python node.

Explore the data here, then choose a distributed engine if the workload grows beyond that node.

What period does this dataset cover?

Convert dof to a date, then inspect the earliest date, latest date, and number of

distinct months.

try_cast returns NULL for values it cannot parse. These results summarize valid

dates; twelve months does not prove every day is present.

```
con.sql(f"""
    WITH flights AS (
        SELECT try_cast(dof AS DATE) AS flight_date
        FROM read_parquet('{FLIGHT_FILES}', union_by_name = true)
    )
    SELECT
        min(flight_date) AS first_day,
        max(flight_date) AS last_day,
        count(DISTINCT date_trunc('month', flight_date)) AS months
    FROM flights
""").show()
```

Which aircraft types dominate?

Knowing a column is populated is only the start. Count the most common aircraft types and see whether a few account for most flights.

The percentages are based on flights with a known typecode, so the top 10 may not add to 100%.

```
con.sql(f"""
    SELECT typecode,
           count(*) AS flights,
           round(100.0 * count(*) / sum(count(*)) OVER (), 2) AS pct
    FROM read_parquet('{FLIGHT_FILES}', union_by_name = true)
    WHERE typecode IS NOT NULL
    GROUP BY typecode
    ORDER BY flights DESC
    LIMIT 10
""").show()
```

Does flight volume change by month?

DuckDB parses the dates and counts flights by month.

We bring those monthly totals into a pandas DataFrame and plot them to spot gaps or spikes worth investigating.

Months with no valid rows are absent from this result; the query does not fill them with zeros.

```
monthly_flights = con.sql(f"""
    WITH flights AS (
        SELECT try_cast(dof AS DATE) AS flight_date
```

```
        FROM read_parquet('{FLIGHT_FILES}', union_by_name = true)
    )
    SELECT date_trunc('month', flight_date) AS month,
           count(*) AS flights
    FROM flights
    WHERE flight_date IS NOT NULL
    GROUP BY date_trunc('month', flight_date)
    ORDER BY month
""").df()

monthly_flights.plot(
    x="month",
    y="flights",
    figsize=(10, 4),
    title="European flights by month",
)
```

Optional: map the question

DuckDB prepares the map data with SQL; Plotly draws the result.

Set RUN_AIRPORT_MAP = True to show up to 200 departure airports with coordinates.

Marker size represents departures, and Barcelona is highlighted in coral.

```
RUN_AIRPORT_MAP = True

if RUN_AIRPORT_MAP:
    import plotly.express as px

    airport_map = con.sql(f"""
        SELECT a.ident,
               a.name,
               a.municipality,
               try_cast(a.latitude_deg AS DOUBLE) AS latitude,
               try_cast(a.longitude_deg AS DOUBLE) AS longitude,
               count(*) AS departures,
               CASE
                   WHEN a.ident = 'LEBL' THEN 'Barcelona'
                   ELSE 'Other airport'
               END AS airport_type
        FROM read_parquet('{FLIGHT_FILES}', union_by_name = true) AS f
        JOIN read_csv_auto('{AIRPORTS}') AS a
          ON f.adep = a.ident
        WHERE a.latitude_deg IS NOT NULL
          AND a.longitude_deg IS NOT NULL
        GROUP BY a.ident, a.name, a.municipality, a.latitude_deg, a.longitude_deg
        ORDER BY departures DESC
        LIMIT 200
    """).df()

    fig = px.scatter_geo(
        airport_map,
        lat="latitude",
        lon="longitude",
        size="departures",
        color="airport_type",
        hover_name="name",
        hover_data={
            "ident": True,
            "municipality": True,
            "departures": ":,",
            "latitude": False,
            "longitude": False,
        },
        color_discrete_map={
            "Barcelona": "#e4572e",
            "Other airport": "#2f6f9f",
        },
        size_max=28,
        scope="europe",
        title="European flight departures, with Barcelona highlighted",
    )

    fig.update_geos(
```

```
        showland=True,
        landcolor="#f2efe8",
        showocean=True,
        oceancolor="#dbeaf2",
        showcountries=True,
        countrycolor="#ffffff",
        lataxis_range=[34, 72],
        lonaxis_range=[-12, 35],
    )

    fig.show()
else:
    print("Airport map skipped. Set RUN_AIRPORT_MAP = True to display it.")
```

Optional: DuckDB's spatial extension

Extensions add capabilities to DuckDB's SQL engine.

INSTALL spatial makes the extension available;

LOAD spatial enables its functions for this connection.

Set RUN_SPATIAL_SQL = True to calculate distances from Barcelona using the airport

reference CSV. The result lists 20 nearby airports, including Barcelona itself at zero.

These are great-circle distances between coordinates. They do not represent actual

flight routes or prove that a flight operated between the airports.

```
RUN_SPATIAL_SQL = True

if RUN_SPATIAL_SQL:
    con.execute("INSTALL spatial")
    con.execute("LOAD spatial")

    con.sql(f"""
        WITH airports AS (
            SELECT ident,
                   name,
                   ST_Point(
                       try_cast(latitude_deg AS DOUBLE),
                       try_cast(longitude_deg AS DOUBLE)
                   ) AS location
            FROM read_csv_auto('{AIRPORTS}')
            WHERE latitude_deg IS NOT NULL
              AND longitude_deg IS NOT NULL
        ),
        barcelona AS (
            SELECT location
            FROM airports
            WHERE ident = 'LEBL'
        )
        SELECT a.ident,
               a.name,
               round(ST_Distance_Sphere(a.location, b.location) / 1000)
                   AS km_from_barcelona
        FROM airports AS a
        CROSS JOIN barcelona AS b
        ORDER BY km_from_barcelona
        LIMIT 20
    """).show()
else:
    print("Spatial SQL skipped. Set RUN_SPATIAL_SQL = True to run the distance example.")
```

8. Take the validated query into your next workflow

Keep the SQL question and verify the results when moving to another engine. DuckDB-specific syntax may need adapting.

For repeatable Fabric runs, add the notebook to a pipeline. Next: 2. Friendly SQL.

2. Friendly SQL

Friendly SQL: familiar queries with less scaffolding

Each example demonstrates one DuckDB SQL convenience. Sections 1-5 form the main walkthrough; sections 6-7 and the bonus cells are additional examples.

Use Pure Python, attach lh_pond, and run 0. Setup with DATA_SCOPE = "one_year". Run this notebook's setup first.

Setup: open DuckDB and choose the flight files

Create this notebook's own connection and set the Parquet path. The examples do not reuse another notebook's Python variables.

f"..." inserts the path into SQL; .show() displays the result. union_by_name = true aligns file columns by name.

```
import duckdb

FLIGHT_FILES = "/lakehouse/default/Files/flights_eu/one_year/flight_list_*.parquet"

con = duckdb.connect()

print("Flight files:", FLIGHT_FILES)
```

Check the demo files

Confirm that the flight folder contains matching Parquet files.

If this check fails, run 0. Setup with DATA_SCOPE = "one_year".

```
from glob import glob

assert glob(FLIGHT_FILES), (
    "Run 0. Setup with DATA_SCOPE = 'one_year' before opening this notebook."
)
print("Demo files found.")
```

1. GROUP BY ALL: Group by the columns you select

Count flights for each origin (adep) and destination (ades) pair.

GROUP BY ALL groups by the selected expressions that are not aggregates.

Here, it replaces GROUP BY adep, ades.

The result shows the ten busiest origin-destination pairs.

```
con.sql(f"""
    SELECT adep,
           ades,
           count(*) AS flights
    FROM read_parquet('{FLIGHT_FILES}', union_by_name = true)
    WHERE adep IS NOT NULL
      AND ades IS NOT NULL
    GROUP BY ALL
    ORDER BY flights DESC
    LIMIT 10
""").show()
```

2. Shape a result with EXCLUDE, REPLACE and RENAME

Start with five columns, drop the flight ID, convert dof to a date and rename the airport fields. Unparseable dates become NULL.

These shortcuts change the query result, not the source files.

```
con.sql(f"""
    WITH flights AS (
        SELECT flt_id, dof, adep, ades, typecode
        FROM read_parquet('{FLIGHT_FILES}', union_by_name = true)
        WHERE adep IS NOT NULL
          AND ades IS NOT NULL
```

```
          AND typecode IS NOT NULL
    )
    SELECT * EXCLUDE (flt_id)
    REPLACE (try_cast(dof AS DATE) AS dof)
    RENAME (adep AS origin_airport, ades AS destination_airport)
    FROM flights
    LIMIT 10
""").show()
```

3. QUALIFY: Keep the top route from each origin

Keep the busiest route for each origin.

row_number() ranks routes; QUALIFY keeps rank 1 without another query layer.

Destination code breaks ties. LIMIT 10 keeps the display short.

```
con.sql(f"""
    SELECT adep,
           ades,
           count(*) AS flights,
           row_number() OVER (
               PARTITION BY adep
               ORDER BY count(*) DESC, ades
           ) AS route_rank
    FROM read_parquet('{FLIGHT_FILES}', union_by_name = true)
    WHERE adep IS NOT NULL
      AND ades IS NOT NULL
    GROUP BY adep, ades
    QUALIFY route_rank = 1
    ORDER BY flights DESC
    LIMIT 10
""").show()
```

4. Alias reuse: name a calculation once

Name the count flights, then reuse it.

DuckDB lets HAVING use the flights alias instead of repeating count(*).

Keep aircraft types with more than 100,000 flights, then sort by that count.

```
con.sql(f"""
    SELECT typecode,
           count(*) AS flights
    FROM read_parquet('{FLIGHT_FILES}', union_by_name = true)
    WHERE typecode IS NOT NULL
    GROUP BY ALL
    HAVING flights > 100000
    ORDER BY flights DESC
""").show()
```

5. Dynamic PIVOT: turn months into columns

Turn month values into columns, with one row per airport.

Use January-March 2025 to keep the result readable on screen.

ON month chooses the column values; USING sum(flights) fills the cells.

DuckDB discovers the month columns from the data. Missing combinations appear as NULL.

```
con.sql(f"""
    PIVOT (
        SELECT adep,
               strftime(try_cast(dof AS DATE), '%Y-%m') AS month,
               count(*) AS flights
        FROM read_parquet('{FLIGHT_FILES}', union_by_name = true)
        WHERE adep IN ('LEBL', 'EDDF', 'EGLL', 'EHAM', 'LFPG')
          AND try_cast(dof AS DATE) >= DATE '2025-01-01'
          AND try_cast(dof AS DATE) < DATE '2025-04-01'
        GROUP BY ALL
    )
    ON month
```

```
    USING sum(flights)
    GROUP BY adep
    ORDER BY adep
""").show()
```

6. Additional example: stack results by column name

Use UNION ALL BY NAME when SELECT columns appear in different orders. Here, role moves from first to last; names keep the output aligned.

ALL retains duplicates. The result shows the ten largest airport-and-role counts.

```
con.sql(f"""
    SELECT 'origin' AS role,
           adep AS airport,
           count(*) AS flights
    FROM read_parquet('{FLIGHT_FILES}', union_by_name = true)
    WHERE adep IS NOT NULL
    GROUP BY ALL

    UNION ALL BY NAME

    SELECT ades AS airport,
           count(*) AS flights,
           'destination' AS role
    FROM read_parquet('{FLIGHT_FILES}', union_by_name = true)
    WHERE ades IS NOT NULL
    GROUP BY ALL

    ORDER BY flights DESC
    LIMIT 10
""").show()
```

7. COLUMNS(...): find related fields

The flight files have several time-related columns. COLUMNS('seen|time') selects

columns whose names contain seen or time, such as first_seen and unix_time.

The | means "or" in this column-name pattern.

This is useful when inspecting a wide file and you do not yet know every field name.

```
con.sql(f"""
    SELECT COLUMNS('seen|time')
    FROM read_parquet('{FLIGHT_FILES}', union_by_name = true)
    LIMIT 10
""").show()
```

Bonus: start with FROM and return a small fraction

Start with the source, then choose the columns.

LIMIT 0.001% limits the result size; it does not select a random sample.

Row order is unspecified. A small result can still require a large scan.

```
con.sql(f"""
    FROM read_parquet('{FLIGHT_FILES}', union_by_name = true)
    SELECT adep,
           ades,
           typecode
    WHERE adep IS NOT NULL
      AND ades IS NOT NULL
      AND typecode IS NOT NULL
    LIMIT 0.001%
""").show()
```

Bonus: profile columns with SUMMARIZE

Inspect column types, ranges, and missing values.

DESCRIBE reports the schema. SUMMARIZE also examines the data.

Keep five profile fields for readability. Text-date ranges do not validate date parsing.

```
con.sql(f"""
    SELECT column_name,
           column_type,
           min,
           max,
           null_percentage
    FROM (
        SUMMARIZE
        SELECT dof, adep, ades, typecode, first_seen, last_seen
        FROM read_parquet('{FLIGHT_FILES}', union_by_name = true)
    )
""").show()
```

Takeaway

DuckDB lets you query Lakehouse files with familiar SQL. These conveniences shorten common analytical queries while keeping the logic visible.

When moving the SQL to another engine, check support for these conveniences and adapt the syntax while preserving the query's meaning.

3. CSV to Parquet

Compare CSV, gzip CSV and Parquet

Store the same January rows three ways, compare file sizes, then ask each copy the same SQL question. This is not a query-speed benchmark.

Use Pure Python, attach lh_pond, and run 0. Setup with DATA_SCOPE = "one_year". Run sections 1-4 in order.

Gzip compresses one CSV file; it is not a ZIP archive.

1. Choose one month of flights

Use January 2025 to keep the conversion manageable.

The source arrives as Parquet; we'll turn it into a CSV text export first.

Conversion path: source Parquet -> plain CSV -> gzip CSV and new Parquet.

```
import duckdb
import os
from pathlib import Path

SOURCE = "/lakehouse/default/Files/flights_eu/one_year/flight_list_202501.parquet"
OUT = "/lakehouse/default/Files/storage_demo"
Path(OUT).mkdir(parents=True, exist_ok=True)

con = duckdb.connect()
rows = con.sql(f"SELECT count(*) FROM read_parquet('{SOURCE}')").fetchone()[0]
print(f"source : {SOURCE}")
print(f"rows   : {rows:,}")
print(f"on disk: {os.path.getsize(SOURCE)/1048576:.1f} MB  (Parquet, as delivered)")
```

1a. Preserve the source types when reading CSV

Build a column-name-to-type dictionary from Parquet's schema. Each CSV reader uses it instead of guessing types, including for large IDs.

Parquet stores these types in the file; CSV does not.

```
# Parquet carries its column names and types.
schema = con.sql(
    f"DESCRIBE SELECT * FROM read_parquet('{SOURCE}')"
).fetchall()

# Build the name-to-type mapping for reading the CSV copies.
column_types = {}

print("columns:")
for name, data_type, *_ in schema:
    column_types[name] = data_type
    print(f"  {name:<22}{data_type}")
```

2. Write the plain CSV export

Use COPY to write the January source rows as CSV with a header. The next two cells read that CSV with the saved types and write compressed copies.

Run all three write cells before comparing the outputs.

```
CSV = f"{OUT}/flights.csv"

con.execute(f"""
    COPY (SELECT * FROM read_parquet('{SOURCE}'))
    TO '{CSV}' (FORMAT CSV, HEADER)
""")
print("Wrote:", CSV)
```

Write the gzip CSV copy

Compress the CSV text, keeping the source column types when reading it.

```
GZIP = f"{OUT}/flights.csv.gz"

con.execute(f"""
    COPY (
        SELECT *
        FROM read_csv('{CSV}', header=true, types={column_types})
    )
    TO '{GZIP}' (FORMAT CSV, HEADER, COMPRESSION GZIP)
""")
print("Wrote:", GZIP)
```

Write the Parquet copy

Read the same CSV and write typed columns with Zstandard compression.

```
PARQUET = f"{OUT}/flights.parquet"

con.execute(f"""
    COPY (
        SELECT *
        FROM read_csv('{CSV}', header=true, types={column_types})
    )
    TO '{PARQUET}' (FORMAT PARQUET, COMPRESSION ZSTD)
""")
print("Wrote:", PARQUET)
```

2a. Check the copies

CSV: supply the saved column types.

Parquet: read the types from the file itself.

All three copies should match the source count.

Matching counts check row totals; they do not prove every value is identical.

```
con.sql(f"""
    SELECT 'Source' AS format, count(*) AS rows
    FROM read_parquet('{SOURCE}')

    UNION ALL

    SELECT 'CSV', count(*)
    FROM read_csv('{CSV}', header=true, types={column_types})

    UNION ALL

    SELECT 'Gzip CSV', count(*)
    FROM read_csv('{GZIP}', header=true, types={column_types})

    UNION ALL

    SELECT 'Parquet', count(*)
    FROM read_parquet('{PARQUET}')
""").show()
```

3. Compare file sizes

Same rows, different storage sizes.

vs CSV = plain CSV size ÷ this file's size.

For example, 5× means one-fifth of the plain CSV size.

Python checks the file sizes; DuckDB handled the conversions.

Compare the measured sizes. Smaller files do not automatically mean faster queries.

```
files = {
    "CSV": CSV,
```

```
    "CSV.gz": GZIP,
    "Parquet (zstd)": PARQUET,
}
csv_size = os.path.getsize(CSV)
print(f"{'format':<20}{'MB':>10}{'vs CSV':>10}")
print("-" * 40)
for label, path in files.items():
    size = os.path.getsize(path)
    print(f"{label:<20}{size / 1048576:>10.1f}{csv_size / size:>9.1f}x")
```

4. Query the plain CSV: Barcelona departures

Count rows where adep = 'LEBL'. The next two cells ask the identical question of gzip CSV and Parquet; all three answers should match.

These cells compare answers, not execution times.

```
con.sql(f"""
    SELECT count(*) AS flights_from_barcelona
    FROM read_csv('{CSV}', header=true, types={column_types})
    WHERE adep = 'LEBL'
""").show()
```

Query the gzip CSV copy

Use the same CSV reader with the compressed file.

```
con.sql(f"""
    SELECT count(*) AS flights_from_barcelona
    FROM read_csv('{GZIP}', header=true, types={column_types})
    WHERE adep = 'LEBL'
""").show()
```

Query the Parquet copy

Change the reader. The file supplies its column types.

```
con.sql(f"""
    SELECT count(*) AS flights_from_barcelona
    FROM read_parquet('{PARQUET}')
    WHERE adep = 'LEBL'
""").show()
```

Optional: inspect the query plan

EXPLAIN shows how DuckDB plans the query.

Look for READ_PARQUET and Filters: adep='LEBL' in the scan.

The ~ row count is an estimate, not the count returned by the query.

This shows the plan, not measured execution times.

The preceding cell showed the answer. Set RUN_EXPLAIN = True to see its plan.

```
RUN_EXPLAIN = True

if RUN_EXPLAIN:
    plan = con.sql(f"""
        EXPLAIN
        SELECT count(*) AS flights_from_barcelona
        FROM read_parquet('{PARQUET}')
        WHERE adep = 'LEBL'
    """).fetchone()[1]

    # Print the plan text so its diagram stays readable.
    print(plan)
else:
    print("Query plan skipped. Set RUN_EXPLAIN = True to display it.")
```

Takeaway

> Convert once. Query again.

Gzip CSV: compressed text that DuckDB can read directly.

Parquet: stored column types and selective column reads.

Use the measured sizes above to compare storage savings for this data.

Keep the original files when you need an archive.

Keep a Parquet copy for repeated SQL analysis.

Inspect the output files, or reset the demo

By default, list file names and sizes in Files/storage_demo.

Set RESET_DEMO_OUTPUT = True only to delete that folder and its contents. Afterwards, restore False and rerun from section 1 to recreate the outputs.

```
from pathlib import Path

OUT = Path("/lakehouse/default/Files/storage_demo")

RESET_DEMO_OUTPUT = False

if RESET_DEMO_OUTPUT:
    import shutil

    # Confirm the resolved target is the demo output folder before deleting it.
    expected_folder = Path("/lakehouse/default/Files").resolve() / "storage_demo"
    if OUT.resolve() != expected_folder:
        raise ValueError("Reset is restricted to Files/storage_demo.")

    if OUT.exists():
        shutil.rmtree(OUT)
        print("Deleted demo output folder:", OUT)

if OUT.exists():
    print("files in", OUT)
    for f in sorted(OUT.iterdir()):
        if f.is_file():
            print(f"  {f.name:<28}{f.stat().st_size / 1048576:>9.1f} MB")
else:
    print("No storage_demo folder. Run sections 1 and 2 to create it.")
```

4. Query Data Where It Lives

Ask the same SQL question of different sources

Count January 2025 departures from Barcelona using a Parquet file, a Delta table and public Blob Storage. Then explore optional shortcut, private Blob and Excel readers.

Use Pure Python, attach lh_pond, and run 0. Setup with DATA_SCOPE = "one_year", including the Delta step. Run the connection cell first; each source defines its own path.

Connect DuckDB: one question, several sources

How many flights departed Barcelona in January 2025? Keep the SELECT and filter; change the reader and source in FROM.

This cell opens the connection used by the examples. The January source counts should match.

```
import duckdb
con = duckdb.connect()
```

1. A file in the Lakehouse

Use the familiar Lakehouse Files path.

read_parquet reads January's file; LEBL selects Barcelona departures.

```
MOUNT = "/lakehouse/default/Files/flights_eu/one_year/flight_list_202501.parquet"

con.sql(f"""
    SELECT count(*) AS flights_from_barcelona
    FROM read_parquet('{MOUNT}')
    WHERE adep = 'LEBL'
""").show()
```

2. A Delta table in the Lakehouse

Setup created dbo.flights_202501 from the January file.

delta_scan uses the transaction log to identify the table's current data files.

INSTALL makes the extension available. LOAD enables it for this connection.

This query reads Delta storage directly; it does not use the SQL endpoint.

```
TABLE = "/lakehouse/default/Tables/dbo/flights_202501"

con.execute("INSTALL delta")
con.execute("LOAD delta")

con.sql(f"""
    SELECT count(*) AS flights_from_barcelona
    FROM delta_scan('{TABLE}')
    WHERE adep = 'LEBL'
""").show()
```

3. A public file in Azure Blob Storage

The httpfs extension reads Parquet over HTTPS.

Upload the January 2025 flight Parquet file downloaded by 0. Setup to your own public Blob container.

Set PUBLIC_BLOB to its HTTPS URL, then set RUN_PUBLIC_BLOB = True. Leave it False to skip.

Use the same dataset to preserve the expected columns and Barcelona flight count.

```
RUN_PUBLIC_BLOB = False
PUBLIC_BLOB = "https://YOUR-STORAGE-ACCOUNT.blob.core.windows.net/YOUR-CONTAINER/flight_list_202501.parquet"

if RUN_PUBLIC_BLOB:
    if "YOUR-" in PUBLIC_BLOB:
        raise ValueError("Set PUBLIC_BLOB to your own public Parquet URL before enabling this example.")
```

```
    con.execute("INSTALL httpfs")
    con.execute("LOAD httpfs")

    con.sql(f"""
        SELECT count(*) AS flights_from_barcelona
        FROM read_parquet('{PUBLIC_BLOB}')
        WHERE adep = 'LEBL'
    """).show()
else:
    print("Public Blob example skipped. Set your URL, then RUN_PUBLIC_BLOB = True.")
```

4. Optional: read the Blob through a OneLake shortcut

Reach the same public Blob file through a Lakehouse Files path. Create blob_flights pointing to your own Blob container containing the January flight file first.

Set RUN_SHORTCUT = True after creating it; leave False to skip.

```
RUN_SHORTCUT = False

if RUN_SHORTCUT:
    SHORTCUT = "/lakehouse/default/Files/blob_flights/flight_list_202501.parquet"

    con.sql(f"""
        SELECT count(*) AS flights_from_barcelona
        FROM read_parquet('{SHORTCUT}')
        WHERE adep = 'LEBL'
    """).show()
else:
    print("Shortcut example skipped. Create blob_flights, then set RUN_SHORTCUT = True.")
```

Optional: read a private Blob with a Key Vault credential

Key Vault supplies a SAS token; DuckDB's Azure extension reads the private file. Configure your vault, secret and Blob address before enabling this example.

Set RUN_PRIVATE_BLOB = True only with secret access and a valid SAS. Leave False to skip; keep the token out of output.

```
RUN_PRIVATE_BLOB = False

if RUN_PRIVATE_BLOB:
    from notebookutils import credentials

    PRIVATE_BLOB = "azure://YOUR-CONTAINER/flight_list_202501.parquet"

    con.execute("INSTALL azure")
    con.execute("LOAD azure")

    # Retrieve the SAS token from Key Vault.
    sas_token = credentials.getSecret(
        "https://YOUR-KEY-VAULT.vault.azure.net/",
        "storage-sas-token",
    ).strip().lstrip("?")

    # Give DuckDB the credential for this storage account.
    con.execute(f"""
        CREATE OR REPLACE SECRET private_flights (
            TYPE azure,
            CONNECTION_STRING

    'BlobEndpoint=https://YOUR-STORAGE-ACCOUNT.blob.core.windows.net/;SharedAccessSignature={sas_token}'
        )
    """)

    # Query the private Parquet file and display the result.
    con.sql(f"""
        SELECT count(*) AS flights_from_barcelona
        FROM read_parquet('{PRIVATE_BLOB}')
        WHERE adep = 'LEBL'
    """).show()

else:
    print("Private Blob example skipped. Set RUN_PRIVATE_BLOB = True after configuring your own resources.")
```

Optional: recover the same total from Excel

Read the workbook created by setup section 7 and sum its flights column. It contains January Barcelona counts by destination, rather than individual flight rows.

Set RUN_EXCEL = True after creating it. read_xlsx handles .xlsx, not .xls; cast the total to BIGINT because Excel numbers are read as doubles.

```
RUN_EXCEL = True

if RUN_EXCEL:
    from pathlib import Path

    EXCEL = "/lakehouse/default/Files/flights_eu/excel/barcelona_departures_202501.xlsx"

    if Path(EXCEL).exists():
        con.execute("INSTALL excel")
        con.execute("LOAD excel")

        con.sql(f"""
            SELECT CAST(sum(flights) AS BIGINT) AS flights_from_barcelona
            FROM read_xlsx('{EXCEL}')
        """).show()
    else:
        print("Workbook not found. Run 0. Setup section 7 to create it.")
else:
    print("Excel example skipped. Set RUN_EXCEL = True to run it.")
```

What changed?

The SQL question stayed the same. The reader, path and access method changed.

| Source | Reader / access |

| --- | --- |

| Lakehouse Parquet | read_parquet |

| Lakehouse Delta | delta_scan reads the transaction log |

| Public Blob | read_parquet over HTTPS with httpfs |

| OneLake shortcut | read_parquet through the Lakehouse Files path |

| Private Blob | read_parquet with Azure extension and Key Vault credential |

| Excel | read_xlsx; sum the pre-aggregated flight counts |

Create a Lakehouse copy when the workflow needs one.

5. Polars SQL Encore

Polars SQL encore: same files, another engine

Additional exercise. Query the same 2025 European flights using Polars SQL in a Pure Python notebook.

Attach lh_pond and run 0. Setup with DATA_SCOPE = "one_year" first. This notebook starts its own SQL context and can run independently of notebooks 1-4.

We will count flights, check coverage, join the airport CSV, and collect a small result. SQL is the main interface; Python registers file scans and displays results.

0. Upgrade Polars before importing it

Run this cell first in a fresh session. It upgrades to the latest compatible Polars 2.x release.

If Polars was already imported, restart the Python session after installation, then continue below. For reproducible results, pin the version printed by the next cell in your Fabric environment.

```
%pip install --upgrade "polars>=2.0,<3"
```

Confirm the version

Polars 2.0 makes streaming the default for lazy queries. Streaming does not guarantee every query fits in memory: joins, sorts, and final results still need resources.

```
import polars as pl
from pathlib import Path

print("Polars:", pl.__version__)
if int(pl.__version__.split(".")[0]) < 2:
    raise RuntimeError("Polars 2.0+ is required. Restart the session after upgrading.")
```

1. Register lazy file scans as SQL tables

scan_parquet and scan_csv describe file sources without loading all rows into a DataFrame.

SQLContext gives those sources table names. .execute() builds a lazy query; .collect() runs it.

The one-year folder keeps the data scope identical to the DuckDB demo. This scan expects the setup dataset's consistent monthly schemas; it does not reproduce DuckDB's union_by_name schema reconciliation.

```
FLIGHT_FILES = "/lakehouse/default/Files/flights_eu/one_year/flight_list_*.parquet"
AIRPORTS = "/lakehouse/default/Files/flights_eu/reference/airports.csv"

if not list(Path(FLIGHT_FILES).parent.glob("flight_list_*.parquet")):
    raise FileNotFoundError("Attach lh_pond and run 0. Setup with DATA_SCOPE = 'one_year'.")
if not Path(AIRPORTS).is_file():
    raise FileNotFoundError(f"Airport reference file missing: {AIRPORTS}")

flights = pl.scan_parquet(FLIGHT_FILES)
airports = pl.scan_csv(
    AIRPORTS,
    schema_overrides={"ident": pl.String, "municipality": pl.String, "name": pl.String},
).select("ident", "municipality", "name")
sql = pl.SQLContext(flights=flights, airports=airports, eager=False)
print("Registered:", sql.tables())
```

2. Count the same flight rows

The familiar SQL stays. Collect only the one-row aggregate result, not all flight records.

For the standard 2025 setup, expect about 16.2 million rows; use the actual result as your source of truth.

```
flight_count = sql.execute("SELECT count(*) AS flights FROM flights").collect()
display(flight_count)
```

3. Inspect the schema and coverage

collect_schema() resolves column names and types, potentially reading metadata. It does not collect every data row.

Then SQL checks non-NULL coverage. Empty strings still count as populated.

```
print(flights.collect_schema())
display(sql.execute("""
    SELECT count(*) AS rows_total,
           round(100.0 * count(adep) / NULLIF(count(*), 0), 1) AS origin_present_pct,
           round(100.0 * count(ades) / NULLIF(count(*), 0), 1) AS destination_present_pct,
           round(100.0 * count(typecode) / NULLIF(count(*), 0), 1) AS typecode_present_pct,
           round(100.0 * count(registration) / NULLIF(count(*), 0), 1) AS registration_present_pct
    FROM flights
""").collect())
```

4. Name Barcelona's busiest routes

The same two left joins add airport names from CSV to flight Parquet. Missing reference matches remain in the result.

We add airport codes to the sort order so tied counts produce a repeatable top ten in both engines.

```
ROUTES_SQL = """
    WITH named_routes AS (
        SELECT f.adep,
               coalesce(dep.municipality, dep.name) AS origin,
               f.ades,
               coalesce(arr.municipality, arr.name) AS destination
        FROM flights AS f
        LEFT JOIN airports AS dep ON f.adep = dep.ident
        LEFT JOIN airports AS arr ON f.ades = arr.ident
        WHERE (f.adep = 'LEBL' OR f.ades = 'LEBL')
          AND f.adep IS NOT NULL
          AND f.ades IS NOT NULL
    )
    SELECT adep, origin, ades, destination, count(*) AS flights
    FROM named_routes
    GROUP BY adep, origin, ades, destination
    ORDER BY flights DESC, adep, ades
    LIMIT 10
"""

barcelona_routes_query = sql.execute(ROUTES_SQL)
barcelona_routes = barcelona_routes_query.collect()
display(barcelona_routes)
```

5. See what Polars plans to do

explain() displays the optimized logical plan without executing the result again.

Look for column projection and filters near the scans. SQL is translated into a Polars lazy plan; this is Polars executing the query.

Open Fabric's Resource usage panel while collecting results to inspect CPU and memory. This notebook is not a speed benchmark.

```
print(barcelona_routes_query.explain())
```

6. Bring back the top 25 airports

The join and aggregation run in Polars. Only the small result is collected into a Polars DataFrame.

The inner join includes airports found in the reference data. Airport IDs keep identically named airports separate.

```
AIRPORTS_SQL = """
    SELECT a.ident, a.municipality, a.name, count(*) AS departures
    FROM flights AS f
    JOIN airports AS a ON f.adep = a.ident
    WHERE f.adep IS NOT NULL
    GROUP BY a.ident, a.municipality, a.name
    ORDER BY departures DESC, a.ident
    LIMIT 25
"""
airport_departures = sql.execute(AIRPORTS_SQL).collect()
display(airport_departures)
```

7. Optional: verify the same answers with DuckDB

Set the flag to True to independently read the same files and compare row counts, Barcelona routes, and busiest airports.

This compares values and row order, not engine-specific integer types or timing. No tables or files are written.

```
RUN_DUCKDB_CHECK = True

if RUN_DUCKDB_CHECK:
    import duckdb

    with duckdb.connect() as con:
        flight_path = FLIGHT_FILES.replace("'", "''")
        airport_path = AIRPORTS.replace("'", "''")
        con.execute(f"CREATE VIEW flights AS SELECT * FROM read_parquet('{flight_path}', union_by_name=true)")
        con.execute(f"CREATE VIEW airports AS SELECT ident, municipality, name FROM
    read_csv_auto('{airport_path}')")
        assert con.sql("SELECT count(*) FROM flights").fetchone()[0] == flight_count.item()
        assert con.sql(ROUTES_SQL).fetchall() == barcelona_routes.rows(), "Barcelona route results differ"
        assert con.sql(AIRPORTS_SQL).fetchall() == airport_departures.rows(), "Airport results differ"
    print("PASS: row count, Barcelona routes, and top 25 airports match DuckDB.")
else:
    print("DuckDB comparison skipped. Set RUN_DUCKDB_CHECK = True to run it.")
```

Takeaway

Same Lakehouse files, familiar SQL, another engine on one Python node.

DuckDB queries file functions directly; here we registered lazy scans and collected only the results. SQL dialects differ, so validate answers when moving a query between engines.

Polars SQL and the expression API share the lazy engine. You can mix them as a workflow grows.

References: [Polars SQL](https://docs.pola.rs/user-guide/sql/intro/) | [Polars 2.0 migration guide](https://docs.pola.rs/releases/upgrade/2/)